

GPS+北斗模块定位显示实验

1.设计需求：

- 1.1 GPS 模块定位数据；
- 1.2 使用 HX4S20 对接收 GPS 定位到的数据，并在 VGA 上进行显示；

2.器件选用：

2.1 GPS 模块：ATK-MO1218 模块

2.1.1 基本参数：

项目	说明
通讯接口	UART
通讯波特率	4800bps、9600bps、19200bps、38400bps（默认）、7600bps、115200bps、230400bps
接收特性	167 通道，支持 QZSS、WAAS、MSAS、EGNOS、GAGAN
定位精度	2.5mCEP（SBAS： 2.0mCEP）
更新速率	1Hz、2Hz、4Hz、5Hz、8Hz、10Hz、20Hz
捕获时间	冷启动： 29S（最快） 温启动： 27S 热启动： 1S
冷启动灵敏度	-148dBm
捕获追踪灵敏度	-165dBm
通讯协议	NMEA-0183 V3.01 SkyTraq binary
工作温度	-40℃~85℃
模块尺寸	25mm*27mm

2.1.2 电气参数

项目	说明
电源电压	DC3.3V~5.0V
IO 口电平	3.3V、5V
功耗	45mA

2.1.3 模块实物图：



2.1.4 引脚说明：

序号	名称	说明
1	VCC	电源（3.3V~5V）
2	GND	电源地
3	TXD	模块串口发送引脚（TTL 电平，不能直接接 RS232 电平！），可接单片机的 RXD
4	RXD	模块串口接收引脚（TTL 电平，不能直接接 RS232 电平！），可接单片机的 TXD
5	PPS	时钟脉冲输出引脚

PPS 引脚说明：

PPS 时钟脉冲输出引脚同时连接到了模块的板载 PPS 指示 LED 上，该引脚是模块板载模组 S1216F8-BD 端口 1PPS 的引脚，该引脚的输出特性可通过程序进行设定。模块板载 PPS 指示 LED（即模块 PPS 时钟脉冲输出引脚）在没有经过程序特殊设定的默认情况下，有两种状态：

- (1) 常亮，此时模块已经开始工作，但还未实现定位。
- (2) 闪烁（100 毫秒灭， 900 毫秒亮）， 此时表示模块已成功定位。

2.2 FPGA 芯片：

核心 FPGA 采用 Intel 公司的 Cyclone 10 系列，型号为 10CL006YU256C8G，FBGA 封装，6772 个逻辑宏单元，2 个锁相环输入脚，256 个引脚的最大可用脚 176 个。

3.协议：

3.1 接收协议：uart

3.2GPS 数据报文协议：

本实验主要识别 GPS 模块输出的 GGA 协议，则无论定位到的何种卫星都识别以下的协议：

(1) GGA

该语句主要包含时间、位置和定位的相关信息，其帧格式如下所示：

```
$--GGA,hhmmss.ss,lll.lll,a,yyyyy.yyy,a,x,uu,v.v,w.w,M,x.x,M,,zzzz*hh<CR><LF>
```

该语句中各个字段的描述，如下表所示：

字段	名称	描述
hhmmss.ss	UTC 时间	定位时的 UTC 时间，格式为 hhmmss.ss，范围为 000000.000~235959.999
lll.lll	纬度	维度，格式为 dddmm.mmmm，高位用 0 填充
A	指示南、北纬	“N”：北纬 “S”：南纬
yyyyy.yyy	经度	经度，格式为 dddmm.mmmm，高位用 0 填充
A	指示东、西经	“E”：东经 “W”：西经
x	指示 GPS 质量	0：未定位 1：已定位，SPS 模式 2：已定位，差分模式
uu	卫星数量	用于定位的卫星数量，范围为 00~24
v.v	HDOP	水平精度因子，范围为 00.0~99.9
w.w	海拔高度	海拔高度，范围为 -9999.99~17999.9，单位为米
x.x	大地水准面高度	单位为米
zzzz	DGPS 站 ID	差分站 ID，范围为 0000~1023（未使用 DGPS 时为 0000）
hh	校验和	

4.FPGA 模块设计：

4.1 PLL 模块：

4.1.1 系统输入时钟 50M

4.1.2 模块输出时钟：

50M: uart 接受模块；

25M: 字符显示模块、VGA 显示模块；

4.2 接收模块：

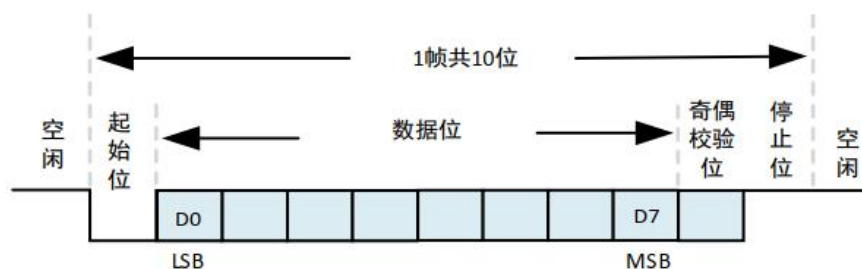
4.2.1 参数：

1) 系统时钟频率：50000000HZ

2) 串口波特率：115200

4.2.2 uart 协议原理：

UART 在发送或接收过程中的一帧数据由 4 部分组成，起始位、数据位、奇偶校验位和停止位



起始位：当不传输数据时，UART 数据传输线通常保持高电压电平。若要开始数据传输，发送 UART 会将传输线从高电平拉到低电平并保持 1 个波特率周期。当接收 UART 检测到

高到低电压跃迁时，便开始以波特率对应的频率读取数据帧中的位。

数据帧：数据帧包含所传输的实际数据。

停止位：为了表示数据包结束，发送 UART 将数据传输线从低电压驱动到高电压并保持 1 到 2 位时间。

4.2.3 接收模块设计思路：

如果使用的是 115200 的波特率，其串口的比特率为 $115200\text{Bps} \times 1\text{bit} = 115200\text{bps}$ 。由计算得串口发送或者接收 1bit 数据的时间为一个波特，即 $1/115200\text{s}$ ，如果用 50MHz（周期为 20ns）的系统时钟来计数，需要计数的个数为 $\text{cnt} = (1\text{s} \times 10^9\text{ns} / 115200\text{bit})\text{ns} / 20\text{ns} \approx 434$ 个系统时钟周期，即每位数据之间的间隔要在 50MHz 的时钟频率下计数 434 次，这样我们就需要一个至少为 9 位的波特率计数器来计数，这里设为 16 位是为了其他波特率的使用。串口是串行发送的，而我们的输出数据是并行输出的，所以还需要一个 4 位的接收数据计数器。我们在系统时钟计数器计数到一半时去采集数据，这时候的数据采集是最正确的。我们将进来的串口接收数据打拍，从而消除亚稳态当检测到串口接收数据的下降沿时，表示接收过程的开始，即 start_flag 信号拉高，此时系统时钟计数器（clk_cnt）开始计数。当系统时钟计数器记到最大值（433）时接收数据计数器（rx_cnt）开始计数，当接收数据计数器大于等于 1 后，计数器每计数一次，将打拍后的数据 uart_rxd_d2 寄存在 rxdata 中。当接收数据计数器记到 9 时，把 rxdata 寄存的数据 rxdata 给到输出端口 uart_data，同时拉高接收完成标志信号 uart_done。这里为什么要记到 9 了，因为我们本次的传输协议是 1bit 起始位，8bit 数据位，1bit 停止位，总共 10bit，而 0~9 总共是计了 10 个数。

```
module uart_rx(
    input          clk          , //系统时钟
    input          rst_n        , //系统复位，低有效
    input          uart_rxd     , //UART 接收端口
    output reg     uart_rx_done, //UART 接收完成信号
    output reg [7:0] uart_rx_data //UART 接收到的数据
);
//parameter define
parameter CLK_FREQ = 50000000;          //系统时钟频率
parameter UART_BPS = 115200 ;           //串口波特率
localparam BAUD_CNT_MAX = CLK_FREQ/UART_BPS; //为得到指定波特率，对系统
时钟计数 BPS_CNT 次
//reg define
reg      uart_rxd_d0;
reg      uart_rxd_d1;
reg      uart_rxd_d2;
reg      rx_flag ; //接收过程标志信号
reg [3:0] rx_cnt   ; //接收数据计数器
reg [15:0] baud_cnt ; //波特率计数器
reg [7:0] rx_data_t ; //接收数据寄存器
//wire define
wire      start_en;

//*****
//**                                     main code
```

```

//*****
//捕获接收端口下降沿(起始位)，得到一个时钟周期的脉冲信号
assign start_en = uart_rxd_d2 & (~uart_rxd_d1) & (~rx_flag);

//针对异步信号的同步处理
always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        uart_rxd_d0 <= 1'b0;
        uart_rxd_d1 <= 1'b0;
        uart_rxd_d2 <= 1'b0;
    end
    else begin
        uart_rxd_d0 <= uart_rxd;
        uart_rxd_d1 <= uart_rxd_d0;
        uart_rxd_d2 <= uart_rxd_d1;
    end
end
//给接收标志赋值
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        rx_flag <= 1'b0;
    else if(start_en) //检测到起始位
        rx_flag <= 1'b1; //接收过程中，标志信号 rx_flag 拉高
        //在停止位一半的时候，即接收过程结束，标志信号 rx_flag 拉低
    else if((rx_cnt == 4'd9) && (baud_cnt == BAUD_CNT_MAX/2 - 1'b1))
        rx_flag <= 1'b0;
    else
        rx_flag <= rx_flag;
end
//波特率的计数器赋值
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        baud_cnt <= 16'd0;
    else if(rx_flag) begin //处于接收过程时，波特率计数器（baud_cnt）进行循环计
数
        if(baud_cnt < BAUD_CNT_MAX - 1'b1)
            baud_cnt <= baud_cnt + 16'b1;
        else
            baud_cnt <= 16'd0; //计数达到一个波特率周期后清零
        end
    else
        baud_cnt <= 16'd0; //接收过程结束时计数器清零
    end
end

```

```

//对接收数据计数器（rx_cnt）进行赋值
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        rx_cnt <= 4'd0;
    else if(rx_flag) begin                //处于接收过程时 rx_cnt 才进行计数
        if(baud_cnt == BAUD_CNT_MAX - 1'b1) //当波特率计数器计数到一个波特率周
期时
            rx_cnt <= rx_cnt + 1'b1;      //接收数据计数器加 1
        else
            rx_cnt <= rx_cnt;
    end
    else
        rx_cnt <= 4'd0;                  //接收过程结束时计数器清零
end

//根据 rx_cnt 来寄存 rxd 端口的数据
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        rx_data_t <= 8'b0;
    else if(rx_flag) begin                //系统处于接收过程时
        if(baud_cnt == BAUD_CNT_MAX/2 - 1'b1) begin //判断 baud_cnt 是否计数到数
据位的中间
            case(rx_cnt)
                4'd1 : rx_data_t[0] <= uart_rxd_d2; //寄存数据的最低位
                4'd2 : rx_data_t[1] <= uart_rxd_d2;
                4'd3 : rx_data_t[2] <= uart_rxd_d2;
                4'd4 : rx_data_t[3] <= uart_rxd_d2;
                4'd5 : rx_data_t[4] <= uart_rxd_d2;
                4'd6 : rx_data_t[5] <= uart_rxd_d2;
                4'd7 : rx_data_t[6] <= uart_rxd_d2;
                4'd8 : rx_data_t[7] <= uart_rxd_d2; //寄存数据的高低位
                default : ;
            endcase
        end
    else
        rx_data_t <= rx_data_t;
    end
    else
        rx_data_t <= 8'b0;
end

//给接收完成信号和接收到的数据赋值
always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin

```

```

        uart_rx_done <= 1'b0;
        uart_rx_data <= 8'b0;
    end
    //当接收数据计数器计数到停止位，且 baud_cnt 计数到停止位的中间时
    else if(rx_cnt == 4'd9 && baud_cnt == BAUD_CNT_MAX/2 - 1'b1) begin
        uart_rx_done <= 1'b1 ; //拉高接收完成信号
        uart_rx_data <= rx_data_t; //并对 UART 接收到的数据进行赋值
    end
    else begin
        uart_rx_done <= 1'b0;
        uart_rx_data <= uart_rx_data;
    end
end
endmodule

```

4.3 解析模块：

根据协议首先我们需要识别报文字头 GGA，确定此报文类型，再识别换行符号结束报文识别。根据逗号将数据分成多个域 cntField，根据协议解析每个域的数据对应顺序的输出。

```

module GGA(
    input          rx_done_toUart, //整个模块由 rx_done_toUart 驱动
    input  [7:0]    rddat_toUart,
    output reg      rx_done,        //接收指令结束，上升沿对齐'n'字符出现时刻，下降沿
    //对齐$xxGGA 后面的','的出现时刻的下一时刻
    output reg [4:0] hh,            //UTC 时间，整数，时 0~24
    output reg [5:0] mm,            //UTC 时间，整数，分 0~59
    output reg [5:0] ss,            //UTC 时间，整数，秒 0~59
    output reg [6:0] ss2,           //UTC 时间，小数，秒 2 位小数，0~99
    output reg [6:0] lat,           //纬度 整数部分，度 0~90
    output reg [5:0] lat2,          //纬度 整数部分，分 0~59
    output reg [16:0] lat3,         //纬度 小数部分，分 5 位小数,0~99999
    output reg      NS,            //区分南北纬，北纬标为 1，南纬标为 0
    output reg [7:0] lon,          //经度 整数部分，度 0~180
    output reg [5:0] lon2,         //经度 整数部分，分 0~59
    output reg [16:0] lon3,        //经度 小数部分，分 5 位小数，0~99999
    output reg      EW,            //区分东西经，东经标为 1，西经标为 0
    output reg [13:0] alt,         //海拔 整数部分，m
    output reg [3:0] alt2          //海拔 小数部分，一位小数 0~9
);
//xxGGA 格式:
$xxGGA,time,lat,NS,lon,EW,quality,numSV,HDOP,alt,altUnit,sep,sepUnit,diffAge,diffStation*cs
<CR><LF>
//time 格式: hhmmss.ss
//lat 格式: ddmn.mmmmm

```

```

//lon 格式: dddmm.mmmmm
//alt 格式: numeric, 一位小数
reg    [4:0]  cntField;      //当前读取第几个域, 以','分隔
reg    [3:0]  cntChar;       //当前读取域中的第几个字符
reg          start  = 1'b0; //NMEA 报文的接收标志, 以$开始, 到\n 结束
reg    [7:0]  charBuffer;
reg    [1:0]  corrNum = 2'd0; //比对是否为 xxGGA, 当 corrNum=3 时, 表示"GGA"字符通过测试, 该条报文即 xxGGA

```

```

wire  [3:0]  num;           //若 charBuffer 为字符 0~9, 则将之转换为数字 0~9
wire          isnum;
reg          afterDot;      //判断是否是"."后面的数字, 在解析海拔时用到
//将字符 0~9 转换为数字 0~9
Char2Num Char2Num_inst(
    .Char(charBuffer),
    .Num(num),
    .isNum(isnum)
);
always @(posedge rx_done_toUart) begin
    charBuffer <= rddat_toUart;

```

```

//-----接收 NMEA 报文数据-----
if(rddat_toUart == "$") begin      //接收到$, 标志着 NMEA 数据的起始
    start      <= 1'b1;
    cntField   <= 5'd0;
    cntChar    <= 4'd0;
    corrNum    <= 2'd0;
end
else if(start) begin
    if(rddat_toUart == "\n") begin //收到\n, 标志 NMEA 报文结束
        start      <= 1'b0;
        rx_done    <= 1'b1;
    end
    else if(rddat_toUart == "," ||
        rddat_toUart == "*") begin //收到','或', 为域的分隔符
        cntField   <= cntField + 1'b1;
        cntChar    <= 4'd0;
    end
    else begin //收到其他字符
        cntChar    <= cntChar + 1'b1;
    end
end
else begin
    start      <= 1'b0;

```

```

    cntField    <= 5'd0;
    cntChar     <= 4'd0;
    corrNum     <= 2'd0;
end
    //-----判断是否为 xxGGA-----
if(cntField == 5'd0) begin
    if(cntChar == 4'd3 && charBuffer == "G") begin
        corrNum <= corrNum + 1'b1;
    end
    else if(cntChar == 4'd4 && charBuffer == "G") begin
        corrNum <= corrNum + 1'b1;
    end
    else if(cntChar == 4'd5 && charBuffer == "A") begin
        corrNum <= corrNum + 1'b1;
    end
end
if(corrNum == 2'd3) begin    //检测到是"xxGGA", 开启解析
    rx_done <= 1'b0;
    corrNum <= 2'b0;
end
    //-----解析 xxGGA-----
if(rx_done == 1'b0) begin
    //解析 UTC 时间
    if(cntField == 5'd1) begin
        if(cntChar == 4'd1) begin        //UTC-hh
            hh <= num*4'd10;
        end
        else if(cntChar == 4'd2) begin
            hh <= hh + num;
        end
        else if(cntChar == 4'd3) begin    //UTC-mm
            mm <= num*4'd10;
        end
        else if(cntChar == 4'd4) begin
            mm <= mm + num;
        end
        else if(cntChar == 4'd5) begin    //UTC-ss
            ss <= num*4'd10;
        end
        else if(cntChar == 4'd6) begin
            ss <= ss + num;
        end
        else if(cntChar == 4'd8) begin    //UTC-.ss
            ss2 <= num*4'd10;
        end
    end
end

```

```

end
else if(cntChar == 4'd9) begin
    ss2 <= ss2 + num;
end
end

```

//解析纬度

```

if(cntField == 5'd2) begin
    if(cntChar == 4'd1) begin //lat-dd
        lat <= num*4'd10;
    end
    else if(cntChar == 4'd2) begin
        lat <= lat + num;
    end
    else if(cntChar == 4'd3) begin //lat-mm
        lat2 <= num*4'd10;
    end
    else if(cntChar == 4'd4) begin
        lat2 <= lat2 + num;
    end
    else if(cntChar == 4'd6) begin //lat.mmmmm
        lat3 <= num;
    end
    else if(cntChar == 4'd7 || cntChar == 4'd8 ||
        cntChar == 4'd9 || cntChar == 4'd10) begin
        lat3 <= lat3*4'd10 + num;
    end
end
end
if(cntField == 5'd3 && cntChar == 4'd1) begin //NS
    if(charBuffer == "N") begin
        NS <= 1'b1;
    end
    else begin
        NS <= 1'b0;
    end
end
end

```

//解析经度

```

if(cntField == 5'd4) begin
    if(cntChar == 4'd1) begin //lon-ddd
        lon <= num;
    end
    else if(cntChar == 4'd2 || cntChar == 4'd3) begin
        lon <= lon*4'd10 + num;
    end
end

```

```

end
else if(cntChar == 4'd4) begin //lon-mm
    lon2    <= num*4'd10;
end
else if(cntChar == 4'd5) begin
    lon2    <= lon2 + num;
end
else if(cntChar == 4'd7) begin //lon-.mmmmm
    lon3    <= num;
end
else if(cntChar == 4'd8 || cntChar == 4'd9 ||
        cntChar == 4'd10 || cntChar == 4'd11) begin
    lon3    <= lon3*4'd10 + num;
end
end
if(cntField == 5'd5 && cntChar == 4'd1) begin //EW
    if(charBuffer == "E") begin
        EW    <= 1'b1;
    end
    else begin
        EW    <= 1'b0;
    end
end
end

//解析海拔
if(cntField == 5'd9) begin
    if(cntChar == 4'd1) begin
        alt        <= num;
        afterDot    <= 1'b0;
    end
    else if(charBuffer==".") begin
        afterDot    <= 1'b1;
        alt2        <= 4'd0;
    end
    else begin
        if(~afterDot) begin
            alt    <= alt*4'd10 + num;    //alt-MMM
        end
        else begin
            alt2    <= alt2*4'd10 + num;    //alt-.M
        end
    end
end
end
end
end

```

```
end
```

```
endmodule
```

```
//END OF parseGGA.v FILE*****
```

4.4 字符显示模块:

设置一个字符的大小为 32*32, 则 640*480 的图像显示 20*15 个字符, 根据图像像素的坐标, 右移 5 位得到字符的坐标 (x_pos, y_pos), 根据坐标显示对应的字符。提前将要显示的字符存在 rom 中, 根据像素坐标和字符坐标得到需要显示的字符的 rom 地址, 进行输出。

显示的字符的个数为 22 位, 采用 RGB565 的形式生成 mif 文件, 则 rom 的深度为 32*32*22=22528, 大小为 16*22528。

```
module char_display(
    input                clk                ,
    input                rst_n              ,
    output [ 11: 0]      video_data         ,//RGB444 颜色数据
    input [ 10: 0]       pixel_xpos         ,//像素点横坐标
    input [ 10: 0]       pixel_ypos         ,//像素点纵坐标
    input                data_req           ,
    input [ 4: 0]        hh                 ,
    input [ 5: 0]        mm                 ,
    input [ 5: 0]        ss                 ,
    input [ 6: 0]        ss2                ,
    input [ 6: 0]        lat                ,
    input [ 5: 0]        lat2               ,
    input [ 16: 0]       lat3               ,
    input                NS                 ,
    input [ 7: 0]        lon                ,
    input [ 5: 0]        lon2               ,
    input [ 16: 0]       lon3               ,
    input                EW                 ,
    input [ 13: 0]       alt                ,
    input [ 3: 0]        alt2               ,
);
//parameter define
//640*480 分辨率时序参数,60fps
wire [4:0] x_pos ;
wire [4:0] y_pos ;
wire [4:0] x_cnt ;
wire [14:0] y_cnt ;
wire [9:0] char_choose;
reg [4:0] char_choose_case;
//字符显示分区
```

```

wire [15:0] video_data_q;
assign x_pos = pixel_xpos >> 'd5 ;
assign y_pos = pixel_ypos >> 'd5 ;
assign x_cnt = pixel_xpos - (x_pos << 'd5) ;
assign y_cnt = pixel_ypos - (y_pos << 'd5) ;
    wire [ 15: 0] rom_addr ;
assign video_data={video_data_q[15:12],video_data_q[10:7],video_data_q[5:1]};

gps_char_rom u_gps_char_rom(
    .address (rom_addr ),
    .clock (clk ),
    .rden (data_req ),
    .q (video_data_q) );
// reg [9:0] addr_cnt;
// always @(posedge clk or negedge rst_n)
//     begin
//         if(!rst_n)
//             addr_cnt <='b0;
//         else if(data_req && addr_cnt <'d703)
//             addr_cnt <= addr_cnt+'b1 ;
//         else if(data_req && addr_cnt == 'd703)
//             addr_cnt <='b0;
//     end

assign char_choose = {y_pos, x_pos} ;
// assign rom_addr = data_req ? ((y_cnt << 'd9) + (y_cnt << 'd7) + (y_cnt << 'd6) +
// addr_cnt ): 'b0;
assign rom_addr = data_req ? ((char_choose_case == 'd22) ? 'b0 : (y_cnt << 'd9) + (y_cnt
<< 'd7) + (y_cnt << 'd6) + (char_choose_case << 'd5) + x_cnt ): 'b0;
// assign rom_addr = ({y_pos, x_pos} == 'b0000000000) ? (y_cnt << 'd9) + (y_cnt << 'd7)
+ (y_cnt << 'd6) + x_cnt : 'b0;
    always @(posedge clk or negedge rst_n)
        begin
            if(!rst_n)
                char_choose_case <='b0;
            else
                case (char_choose)
                    'b0000000000:
                        char_choose_case <='d10;
                    'b0000000001:
                        char_choose_case <='d11;
                    'b0000000010:
                        char_choose_case <= hh / 'd10 ;
                    'b0000000011:

```

```
char_choose_case<= hh % 'd10;  
'b0000000100:  
char_choose_case<='d21;  
'b0000000101:  
char_choose_case<=mm / 'd10;  
'b0000000110:  
char_choose_case<=mm % 'd10;  
'b0000000111:  
char_choose_case<='d21;  
'b0000001000:  
char_choose_case<=ss / 'd10;  
'b0000001001:  
char_choose_case<=ss % 'd10;  
'b0000001010:  
char_choose_case<='d21;  
'b0000001011:  
char_choose_case<=ss2 / 'd10;  
'b0000001100:  
char_choose_case<=ss2 % 'd10;  
'b0000100000:  
char_choose_case<='d12;  
'b0000100001:  
char_choose_case<='d13;  
'b0000100010:  
char_choose_case<= lat / 'd10 ;  
'b0000100011:  
char_choose_case<= lat % 'd10;  
'b0000100100:  
char_choose_case<='d21;  
'b0000100101:  
char_choose_case<=lat2 / 'd10;  
'b0000100110:  
char_choose_case<= lat2 % 'd10;  
'b0000100111:  
char_choose_case<='d21;  
'b0000101000:  
char_choose_case<=(lat3 >>'d10) / 'd10;  
'b0000101001:  
char_choose_case<=(lat3 >>'d10) % 'd10;  
'b0001000000:  
char_choose_case<='d15;  
'b0001000001:  
char_choose_case<='d16;  
'b0001000010:
```

```

char_choose_case<= NS ? 'd15 : 'd16;
'b0001100000:
char_choose_case<='d14;
'b0001100001:
char_choose_case<='d13;
'b0001100010:
char_choose_case<= lon / 'd100 ;
'b0001100011:
char_choose_case<= lon % 'd100 / 'd10;
'b0001100100:
char_choose_case<= lon % 'd10;
'b0001100101:
char_choose_case<=lon2 / 'd10;
'b0001100110:
char_choose_case<=lon2 % 'd10;
'b0001100111:
char_choose_case<='d21;
'b0001101000:
char_choose_case<=(lon3 >>'d10) / 'd10;
'b0001101001:
char_choose_case<=(lon3 >>'d10) % 'd10;
'b0010000000:
char_choose_case<='d17;
'b0010000001:
char_choose_case<='d18;
'b0010000010:
char_choose_case<= EW ? 'd17 : 'd18;
'b0010100000:
char_choose_case<='d19;
'b0010100001:
char_choose_case<='d20;
'b0010100010:
char_choose_case<= (alt >> 'd10) / 'd10 ;
'b0010100011:
char_choose_case<= (alt >> 'd10) % 'd10;
'b0010100100:
char_choose_case<=alt % 'd1000/'d100;
'b0010100101:
char_choose_case<=alt % 'd100 / 'd10;
'b0010100110:
char_choose_case<=alt % 'd10;
'b0010100111:
char_choose_case<='d21;
'b0010101000:

```

```

        char_choose_case<=alt2 / 'd10;
        default:
        char_choose_case<='d22;
        // char_choose_case<='d0;
    endcase
end

endmodule

```

引脚锁定：

根据 doc-->gps_display.tcl 文件配置引脚，配置如下：

```

package require ::quartus::project
set_location_assignment PIN_E15 -to CLK_50
set_location_assignment PIN_M1 -to reset_n
set_location_assignment PIN_P16 -to video_hs
set_location_assignment PIN_N15 -to video_vs
set_location_assignment PIN_N16 -to video_rgb[0]
set_location_assignment PIN_L15 -to video_rgb[1]
set_location_assignment PIN_F16 -to video_rgb[10]
set_location_assignment PIN_F15 -to video_rgb[11]
set_location_assignment PIN_L16 -to video_rgb[2]
set_location_assignment PIN_K15 -to video_rgb[3]
set_location_assignment PIN_K16 -to video_rgb[4]
set_location_assignment PIN_J15 -to video_rgb[5]
set_location_assignment PIN_J16 -to video_rgb[6]
set_location_assignment PIN_J11 -to video_rgb[7]
set_location_assignment PIN_G16 -to video_rgb[8]
set_location_assignment PIN_G15 -to video_rgb[9]
set_location_assignment PIN_E7 -to UART_RXD

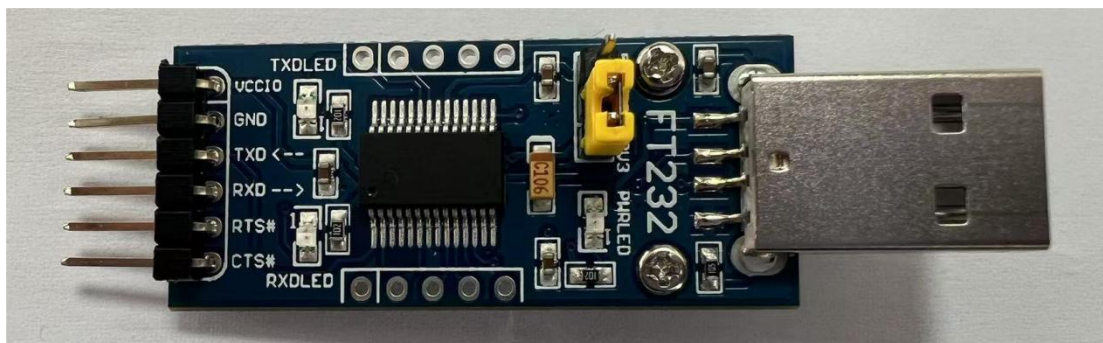
```

5.实验结果：

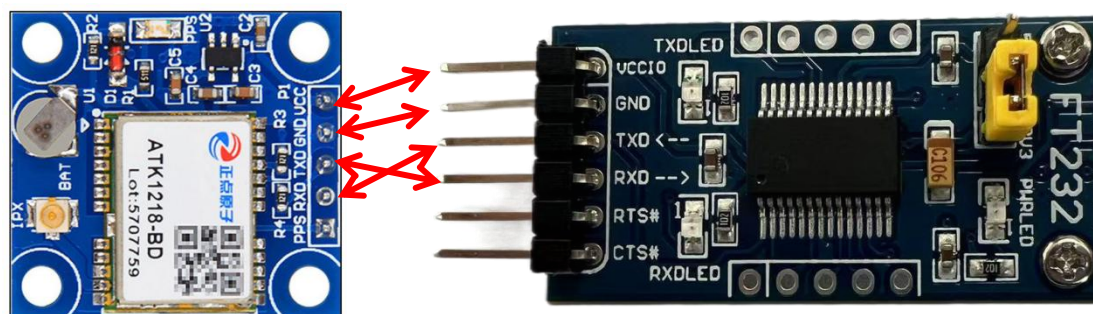
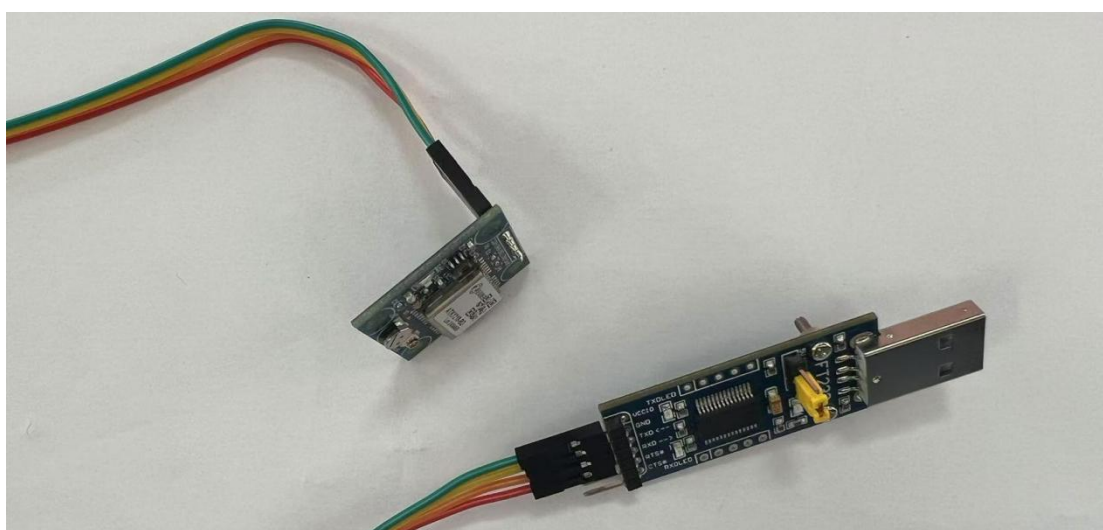
5.1 上位机配置 GPS 模块：

GNSS_View 软件可以设置 ATK-MO1218 模块的串口通讯波特率，出厂默认情况下 ATK-MO1218 模块的串口通讯波特率为 34800bps，下面演示使用 GNSS_View 软件设置 ATK-MO1218 模块的串口通讯波特率为 115200bps，并保存至 Flash 中。点击 GNSS_View 软件菜单栏中的 Binary→Configure Serial Port，便可看到弹出的串口配置窗口，根据需求配置 flash，首先串口连接模块，如下图所示：

- (1) 准备一个 USB 转串口模块



(2) 用杜邦线将转串口模块与 GPS 模块连接



注意：VCC 与 GND 不要接错，USB 串口模块的 RXD 接 GPS 模块的 TXD，USB 串口模块的 TXD 接 GPS 模块的 RXD，GPS 模块接 5v 或 3.3v 均可。

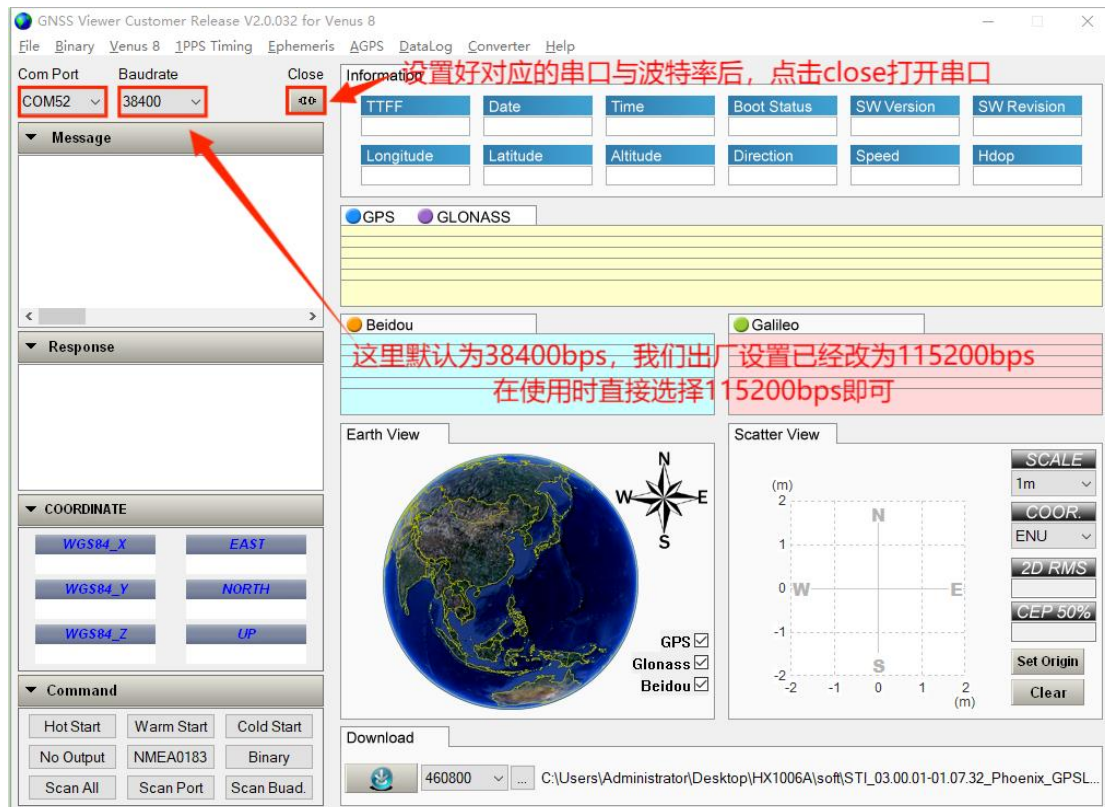
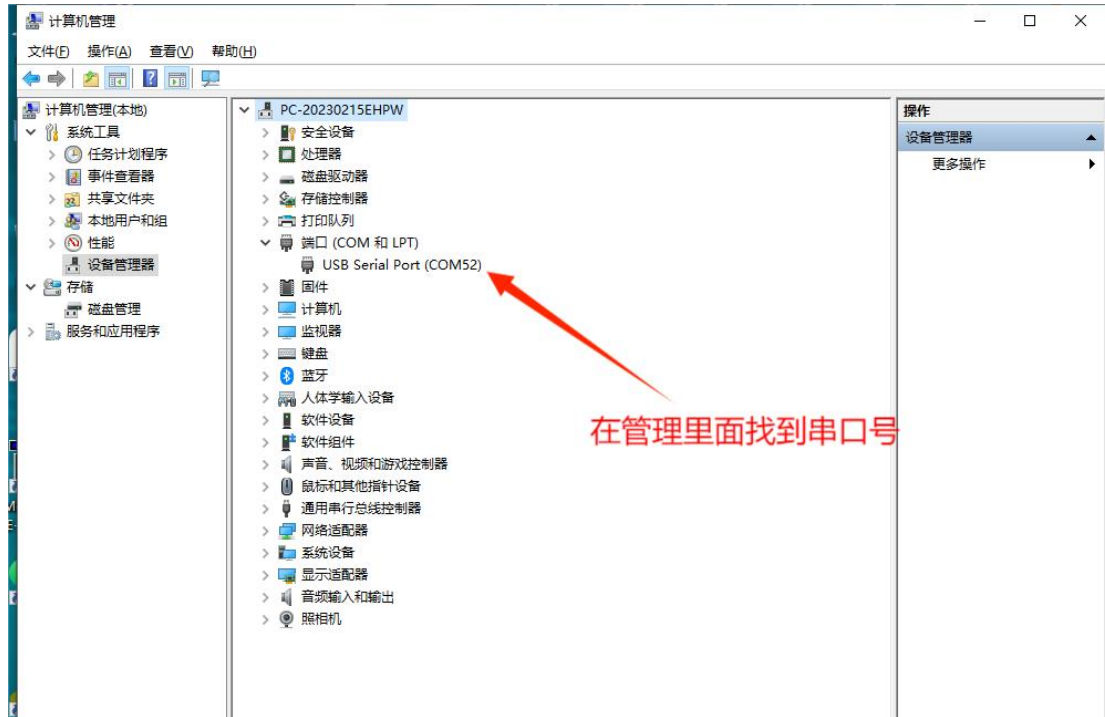
(3) 将 GPS 模块插上天线，另一端 USB 转串口模块插在电脑 USB 口上



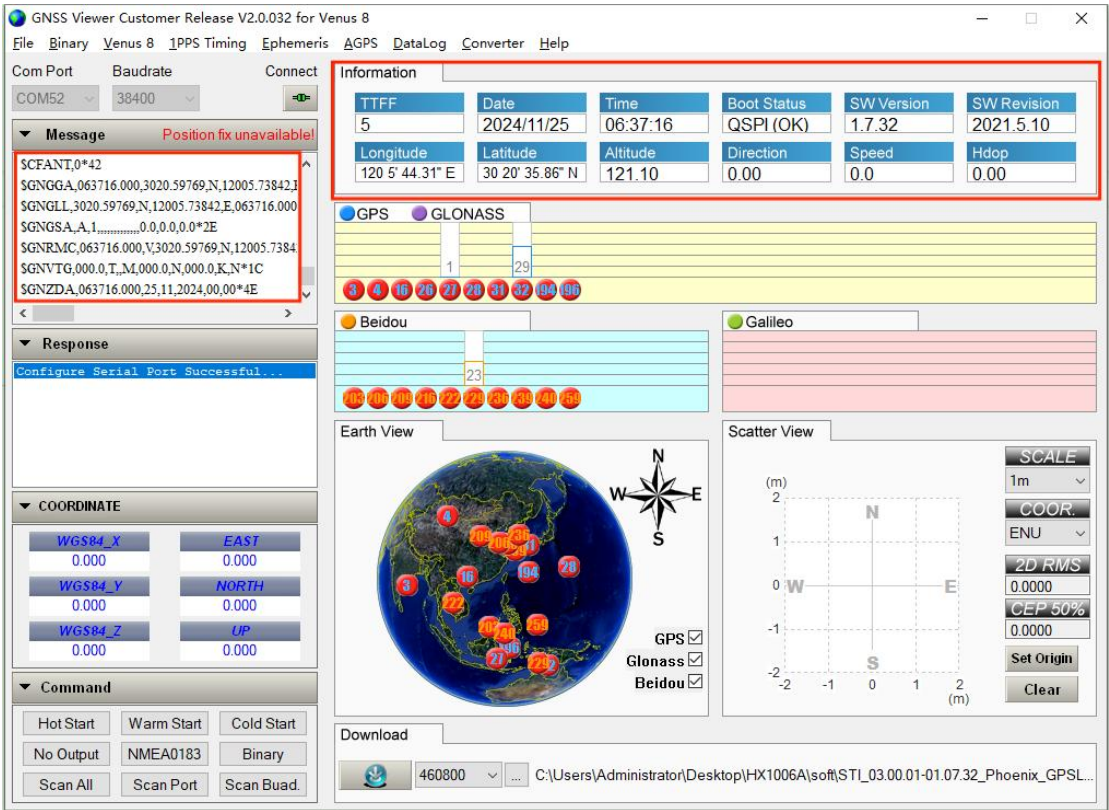
(4) 打开上位机 GNSS_Viewer-7.0.2.1 修改波特率

GNSS_Viewer-7.0.2.1.exe 2021/11/9 16:32 应用程序 1,380 KB

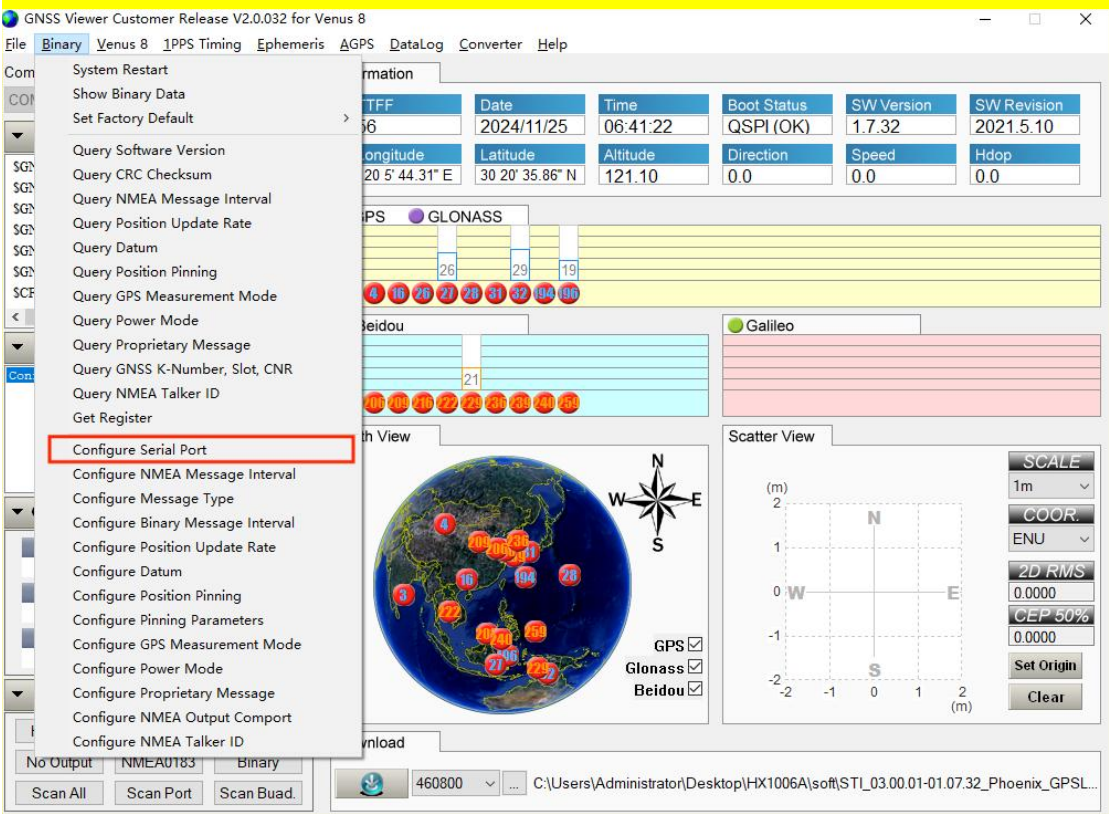
GNSS_Viewer 软件可以设置 ATK-MO1218 模块的串口通讯波特率，出厂默认情况下 ATK-MO1218 模块的串口通讯波特率为 34800bps，（模块已经经过测试，波特率已经设置为 115200bps）下面演示使用 GNSS_Viewer 软件设置 ATK-MO1218 模块的串口通讯波特率为 115200bps，



打开串口后，我们可以看到各种打印信息与坐标信息等等

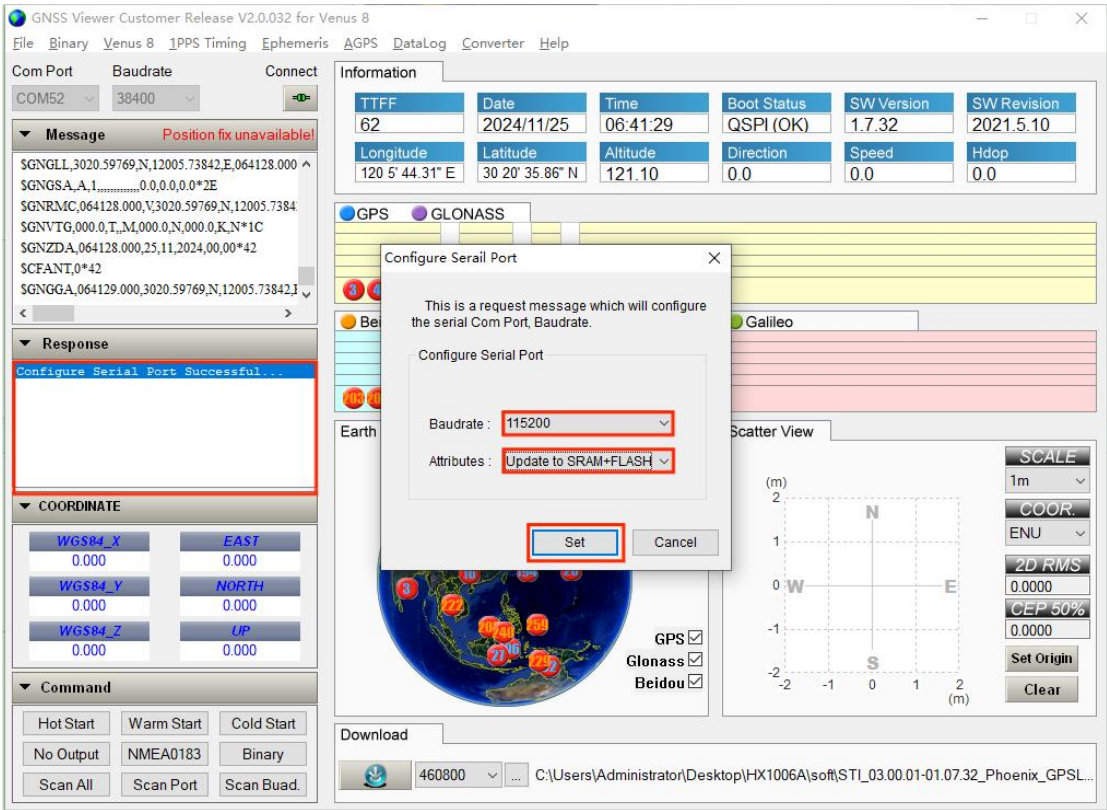


点击 GNSS_Viewer 软件菜单栏中的 Binary→Configure Serial Port

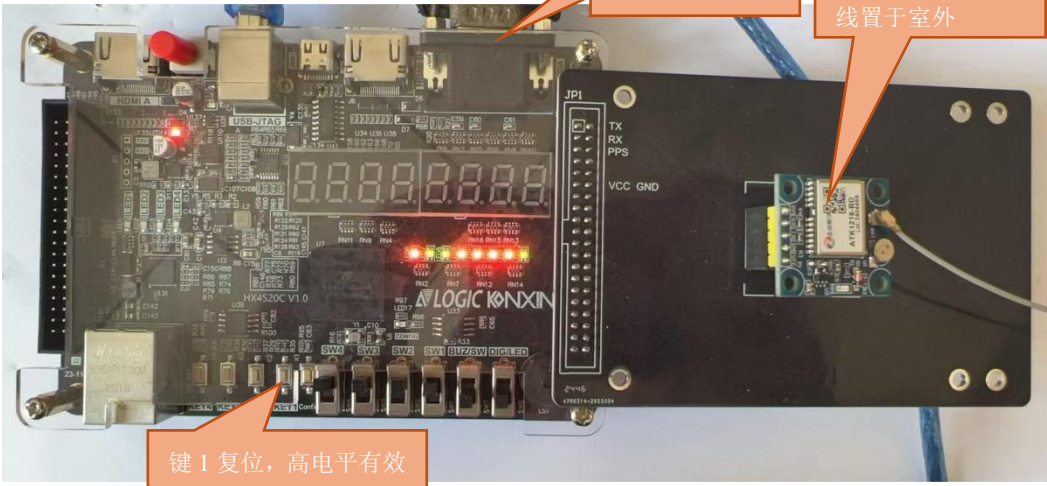


修改波特率为 115200bps，选择保存到 FLASH 中后点击 Set，至此就完成了串口速率的

修改，修改成功后会在 Response 显示 Configure Serial Port Successful...表示修改成功



5.2 上板验证:



5.2.2: 显示结果:

时间 06. 59. 36. 00
纬度 30. 20. 58
N S N
经度 12005. 72
E W E
海拔 00121. 0